

Log Analysis for web security – Practical Report

Amir Askhatov

Step 1: Dataset Collection

-Training Data: dec_2016.csv

-Testing Data: feb_2017.csv

Step 2: Importing Required libraries

```
import sys
import numpy as np
from sklearn import tree, linear_model
import argparse
import re
import pandas as pd
```

Step 3: Reading the Dataset

```
# Define names of of columns
data_columns = ['url_len', 'parameter', 'return_code', 'labels',
                'Http_req']

# reading training data
train_data = pd.read_csv(path_to_train_data, names = data_columns)
# reading test data
test_data = pd.read_csv(path_to_test_data, names = data_columns)
```

	url_len	parameter	return_code	labels	Http_req
0	47	1	301	0	GET /self.logs/error.log.2015-10-28.gz HTTP/1.1
1	38	1	404	0	GET /access.log.2015-02-18.gz HTTP/1.1
2	47	1	200	0	GET /self.logs/error.log.2015-10-11.gz HTTP/1.1
3	47	1	200	0	GET /self.logs/error.log.2015-12-25.gz HTTP/1.1
4	38	1	404	0	GET /access.log.2015-02-21.gz HTTP/1.1

Step 4: Exploring the data

Printed first 5 rows of train and test data using `head()`.

Verified that labels are 0 or 1.

Step 5: Preprocessing (Feature and Label Extraction)

```
# Define names of of columns
data_columns = ['url_len', 'parameter', 'return_code', 'labels', 'Http_req']

# reading training data
train_data = pd.read_csv(path_to_train_data, names = data_columns)
# reading test data
test_data = pd.read_csv(path_to_test_data, names = data_columns)

# print labels for training and test data

print("The Training Label Data is Shown below") #print statement
print(train_data['labels']) #printing the training data labels
print("-----") #print statement for simple formatting

print("The Testing Label Data is Shown below") #printing label
print(test_data['labels']) ##printing the testing data labels
print("-----") #print statement simple formatting
```

```

.. The Training Label Data is Shown below
47    0
38    0
47    0
47    0
38    0
..
47    0
47    0
48    0
47    0
48    0
Name: labels, Length: 1747, dtype: int64
-----
The Testing Label Data is Shown below
47    0
47    0
47    0
48    0
48    0
..
47    0
47    0
48    0
47    0
47    0
Name: labels, Length: 1876, dtype: int64
-----

```

Step 6: Train Decision Classifier

```

attack_classifier = tree.DecisionTreeClassifier()
attack_classifier = attack_classifier.fit(training_features, training_labels)

```

```
# Get training features and labels
training_features = train_data.iloc[:, [0,1,2]] # they
training_labels = train_data['labels'] #extracting tra

# Get testing features and labels
testing_features = test_data.iloc[:, [0,1,2]] # they ar
testing_labels = test_data['labels'] #extracting testi
```

```
# Remove the class labels from the features

# # Drop labels from training features
# training_features = training_features.drop('labels',

# # Dropping from testing features
# testing_features = testing_features.drop('labels', a
```

```
#printing training data
training_features
```

	url_len	parameter	return_code
0	47	1	301
1	38	1	404
2	47	1	200
3	47	1	200
4	38	1	404
...	...	...	...
1742	47	1	200
1743	47	1	200
1744	48	1	200
1745	47	1	301
1746	48	1	404

1747 rows x 3 columns

Step 7: Make Predictions

We predict labels for the test dataset:

```
Predictions = attack_classifier.predict(testing_features)
```

```
Print(predictions[:10])
```

```
Training features sample:
  url_len  parameter  return_code
0      47          1          301
1      38          1          404
2      47          1          200
3      47          1          200
4      38          1          404
Training labels sample:
0      0
1      0
2      0
3      0
4      0
Name: labels, dtype: int64
Testing features sample:
  url_len  parameter  return_code
0      47          1          200
1      47          1          200
2      47          1          200
3      48          1          200
4      48          1          200
Testing labels sample:
0      0
1      0
2      0
...
===== Decision Tree Classifier =====

Sample predictions:
[0 0 0 0 0 0 0 0 0 0]

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

Step 8: Model Evaluation

```
from sklearn.metrics import confusion_matrix, classification_report #import
classification report and confusion matrix

print("The confusion matrix is shown below") #print statement
print(confusion_matrix(testing_labels, predictions)) #prints the confusion matrix
print("-----") #print statement for simple
formatting

print("The classification report is shown below") #print statement
print(classification_report(testing_labels, predictions, labels=[0, 1])) #prints
the classification report, there are only two labels, either attack or not attack
```

```
print("-----") #print statement for simple  
formatting
```

The confusion matrix is shown below

```
[[1802  35]  
 [  10  29]]
```

The classification report is shown below

	precision	recall	f1-score	support
0	0.99	0.98	0.99	1837
1	0.45	0.74	0.56	39
accuracy			0.98	1876
macro avg	0.72	0.86	0.78	1876
weighted avg	0.98	0.98	0.98	1876

Conclusion:

The Decision Tree model was successfully implemented and evaluated. The final confusion matrix and classification report suggest that the model performed well on detecting normal request, but could be further improved on detecting attacks due to class imbalance.